



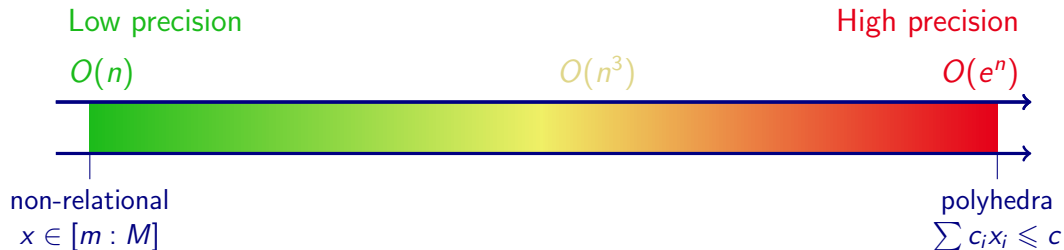
A Lattice of Union-Finds

SAS – Oakland, California, USA

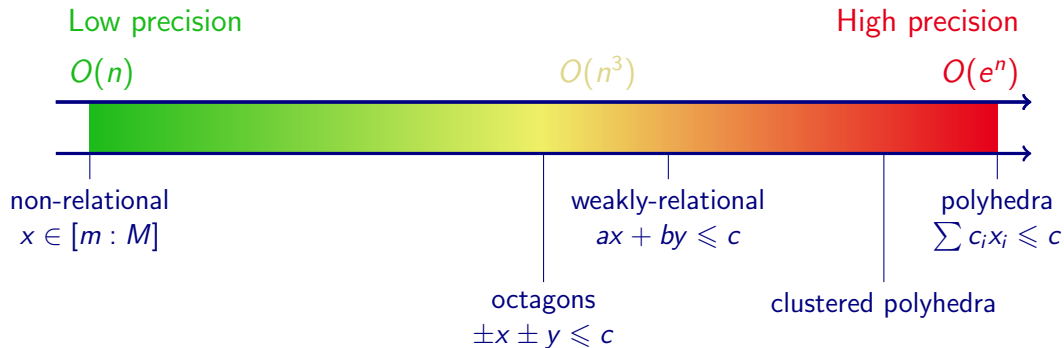
Dorian Lesbre, Matthieu Lemerre
Université Paris-Saclay, CEA List

October 6th, 2026

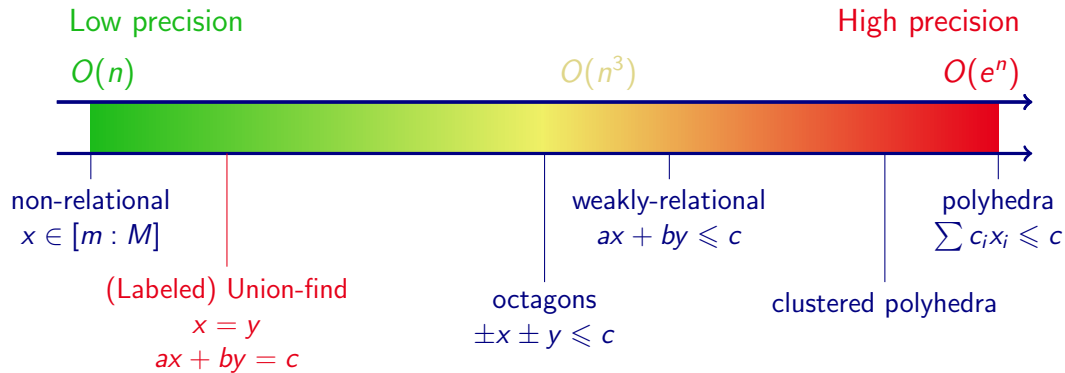
A fast equality domain



A fast equality domain

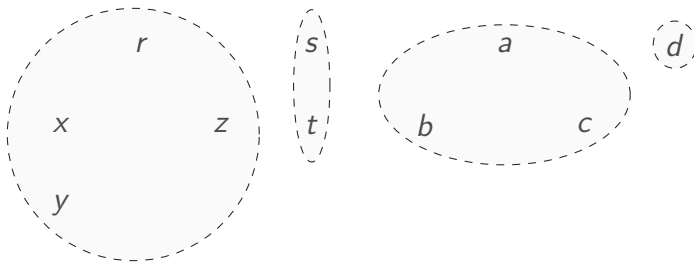


A fast equality domain



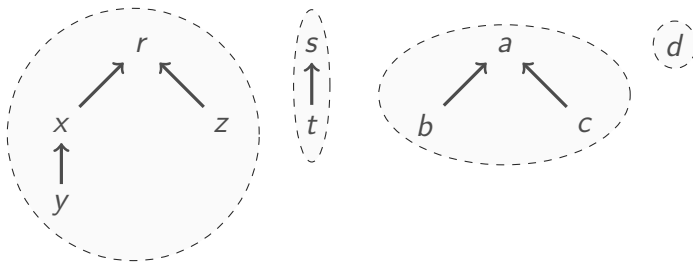
Union-find data structure

Union-find represents a partition/equivalence classes



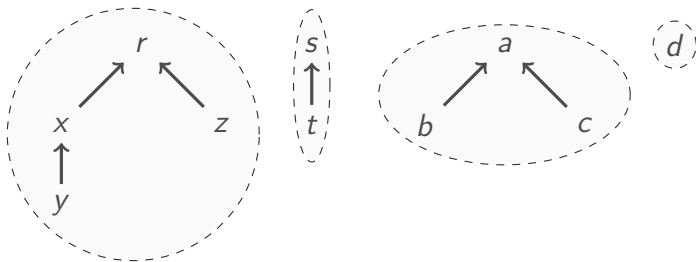
Union-find data structure

Union-find represents a partition/equivalence classes using a rooted forest



Union-find data structure

Union-find represents a partition/equivalence classes using a rooted forest

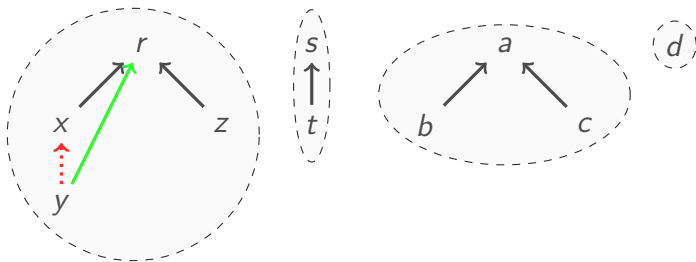


Operations:

- **find** : *node* $\rightarrow$ *node* returns the root: $\text{find } y = r$

Union-find data structure

Union-find represents a partition/equivalence classes using a rooted forest

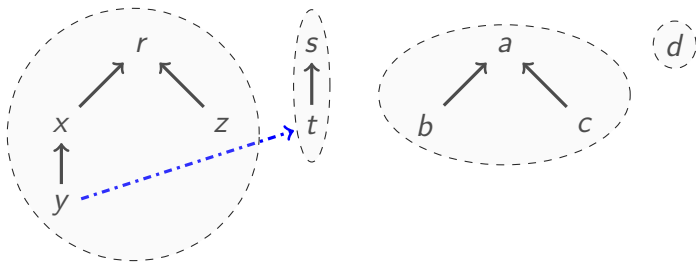


Operations:

- **find** : *node* $\rightarrow$ *node* returns the root: $\text{find } y = r$ and shortens path

Union-find data structure

Union-find represents a partition/equivalence classes using a rooted forest

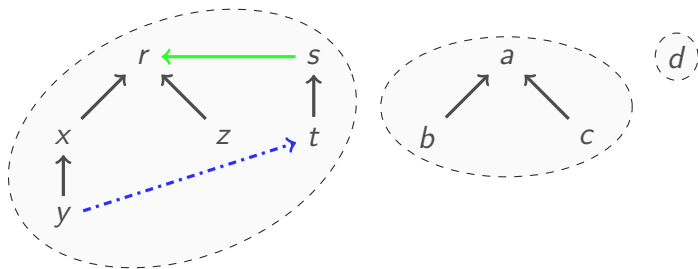


Operations:

- **find** : $node \rightarrow node$ returns the root: $\text{find } y = r$ and shortens path
- **union** : $node \rightarrow node \rightarrow unit$ adds an equality: $\text{union } y \ t$

Union-find data structure

Union-find represents a partition/equivalence classes using a rooted forest



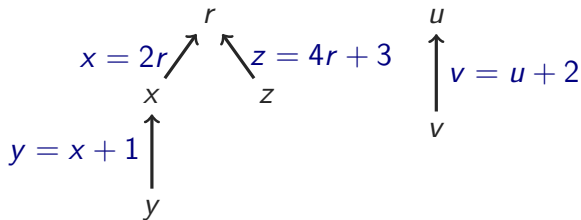
Operations:

- **find** : *node* $\rightarrow$ *node* returns the root: find *y* = *r* and shortens path
- **union** : *node* $\rightarrow$ *node* $\rightarrow$ *unit* adds an equality: union *y* *t*

Prior experience: PLDI 2025

A fast relational domain based on union-find:

- Used an extension of UF: **labeled union-find**

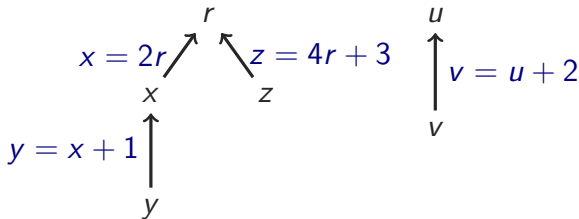


Can represent **injective relations** (ex: $y = ax + b$), not just equality

Prior experience: PLDI 2025

A fast relational domain based on union-find:

- Used an extension of UF: **labeled union-find**

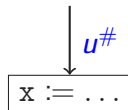


Can represent **injective relations** (ex: $y = ax + b$), not just equality

- Limited to **flow-insensitive** relations, used in SSA analysis

Problem and contributions

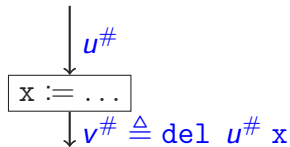
To use union-find to store **flow-sensitive information**, we must:



Problem and contributions

To use union-find to store **flow-sensitive information**, we must:

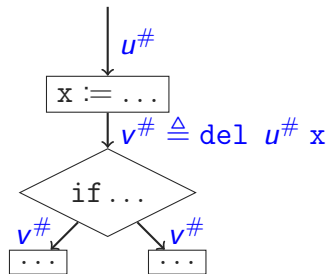
- Remove equalities as values mutate



Problem and contributions

To use union-find to store **flow-sensitive information**, we must:

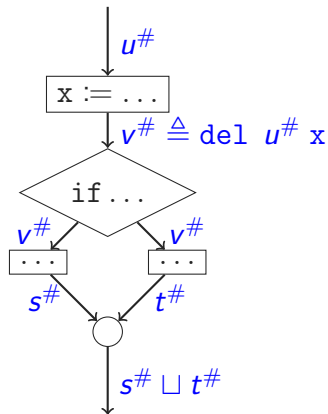
- Remove equalities as values mutate
- Copy at control-flow splits



Problem and contributions

To use union-find to store **flow-sensitive information**, we must:

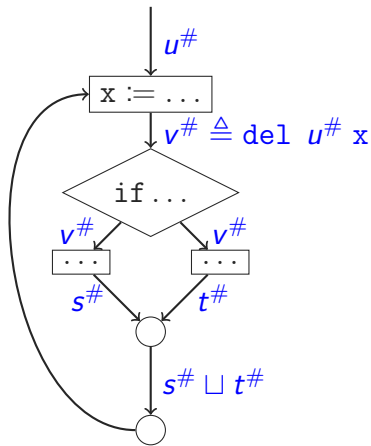
- **Remove equalities** as values mutate
- **Copy** at control-flow splits
- **Join** at control-flow joins



Problem and contributions

To use union-find to store **flow-sensitive information**, we must:

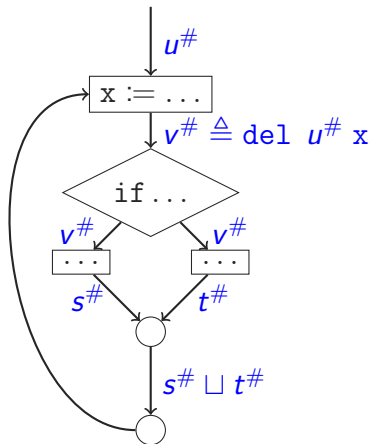
- **Remove equalities** as values mutate
- **Copy** at control-flow splits
- **Join** at control-flow joins
- **Inclusion test** and **widening** at loop heads



Problem and contributions

To use union-find to store **flow-sensitive information**, we must:

- **Remove equalities** as values mutate
Solution: use a union-find with deletion
Not needed for SSA analysis
- **Copy** at control-flow splits
Solution: functional map/persistent array
- **Join** at control-flow joins
- **Inclusion test** and **widening** at loop heads



Problem and contributions

To use union-find to store **flow-sensitive information**, we must:

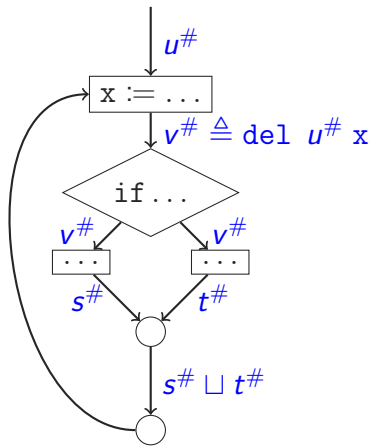
- **Remove equalities** as values mutate
Solution: use a union-find with deletion
Not needed for SSA analysis
- **Copy** at control-flow splits
Solution: functional map/persistent array
- **Join** at control-flow joins

Contribution: fast join algorithm

- **Inclusion test** and **widening** at loop heads

Contribution: fast inclusion test

No widening (finite-height lattice)

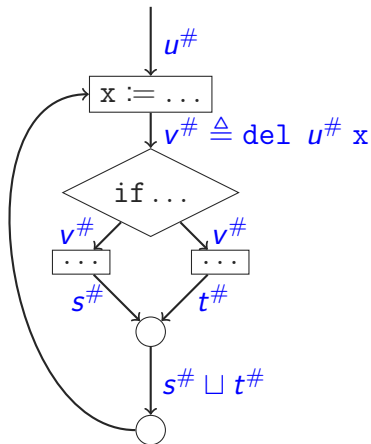


Problem and contributions

To use union-find to store **flow-sensitive information**, we must:

- **Remove equalities** as values mutate
Solution: use a union-find with deletion
Not needed for SSA analysis
- **Copy** at control-flow splits
Solution: functional map/persistent array
- **Join** at control-flow joins
Contribution: fast join algorithm
- **Inclusion test** and **widening** at loop heads
Contribution: fast inclusion test
No widening (finite-height lattice)

We also provide a **meet algorithm**, making union-find a lattice

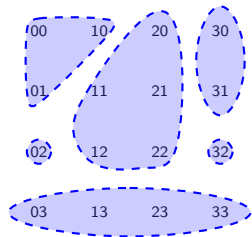




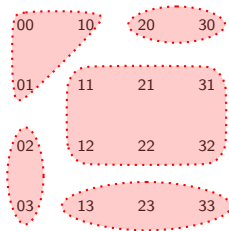
1. Join algorithm

Joining union-finds

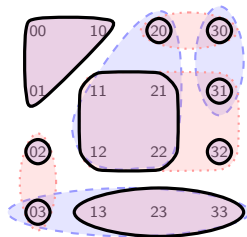
Join is **equivalence class intersection**:



U



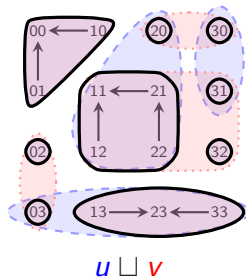
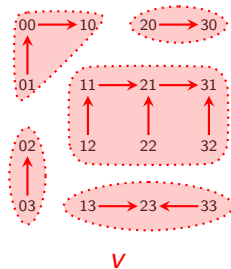
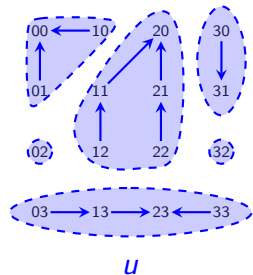
V



$U \sqcap V$

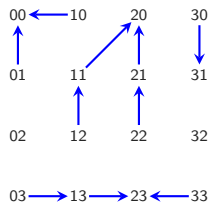
Joining union-finds

How do we compute it from parent arrows?

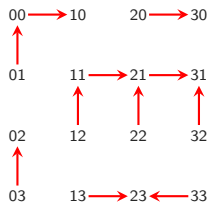


Joining union-finds

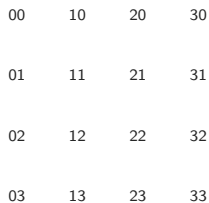
Identify an intersection by the **representative pair** ($\bullet$, $\bullet$):



U



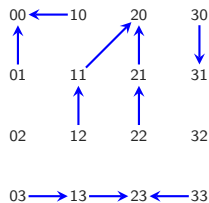
V



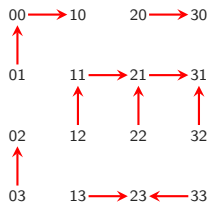
$U \sqcup V$

Joining union-finds

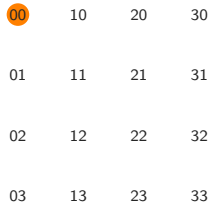
Identify an intersection by the **representative pair** ($\bullet$, $\bullet$):
Linearly scan through all nodes



U



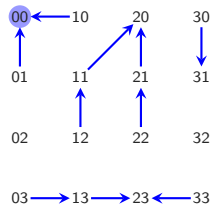
V



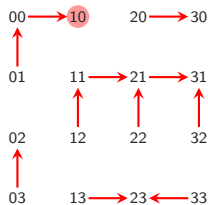
$U \sqcup V$

Joining union-finds

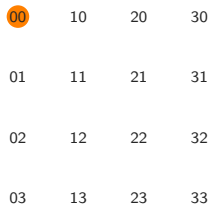
Identify an intersection by the **representative pair** $(\bullet, \bullet)$:
Linearly scan through all nodes



U



V

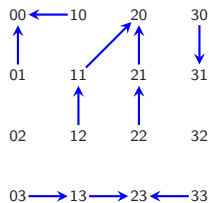


$U \sqcup V$

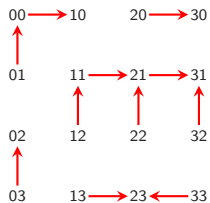
$(00, 10) \mapsto 00$

Joining union-finds

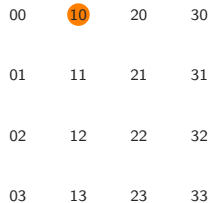
Identify an intersection by the **representative pair** $(\bullet, \bullet)$:
Linearly scan through all nodes



U



V

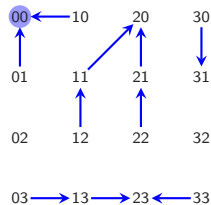


$U \sqcup V$

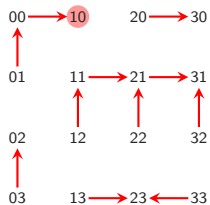
$(00, 10) \mapsto 00$

Joining union-finds

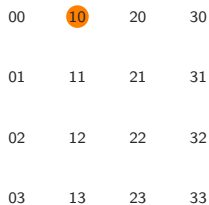
Identify an intersection by the **representative pair** $(\bullet, \bullet)$:
Linearly scan through all nodes



U



V

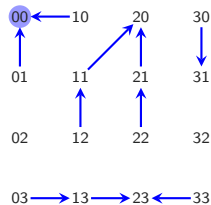


$U \sqcup V$

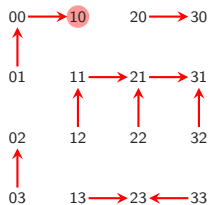
$(00, 10) \mapsto 00$

Joining union-finds

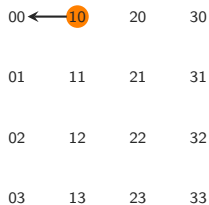
Identify an intersection by the **representative pair** $(\bullet, \bullet)$:
Union nodes which map to the same pair



U



V

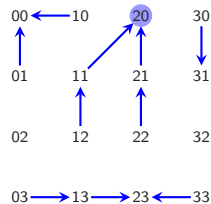


$U \sqcup V$

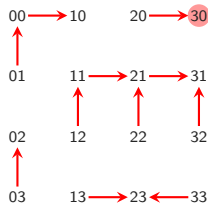
$(00, 10) \mapsto 00$

Joining union-finds

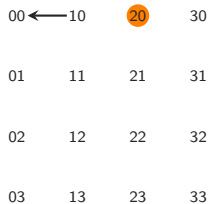
Identify an intersection by the **representative pair** $(\bullet, \bullet)$:
Union nodes which map to the same pair



U



V

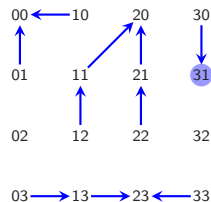


$U \sqcup V$

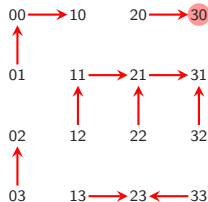
$(00, 10) \mapsto 00$
 $(20, 30) \mapsto 20$

Joining union-finds

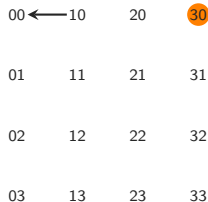
Identify an intersection by the **representative pair** $(\bullet, \bullet)$:
 Union nodes which map to the same pair



U



V

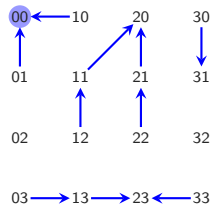


$U \sqcup V$

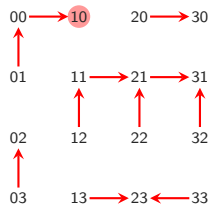
$(00, 10) \mapsto 00$
 $(20, 30) \mapsto 20$
 $(31, 30) \mapsto 30$

Joining union-finds

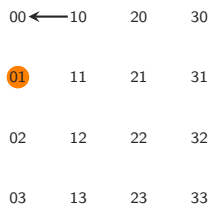
Identify an intersection by the **representative pair** $(\bullet, \bullet)$:
Union nodes which map to the same pair



U



V

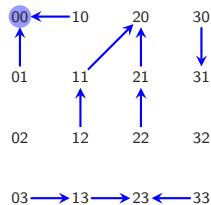


$U \sqcup V$

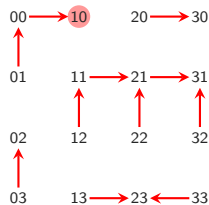
$(00, 10) \mapsto 00$
 $(20, 30) \mapsto 20$
 $(31, 30) \mapsto 30$

Joining union-finds

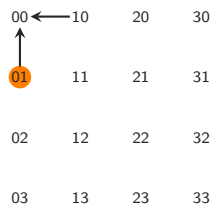
Identify an intersection by the **representative pair** $(\bullet, \bullet)$:
Union nodes which map to the same pair



U



V

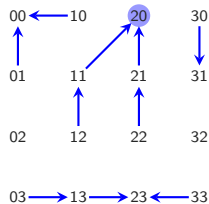


$U \sqcup V$

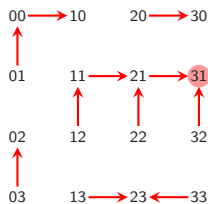
$(00, 10) \mapsto 00$
 $(20, 30) \mapsto 20$
 $(31, 30) \mapsto 30$

Joining union-finds

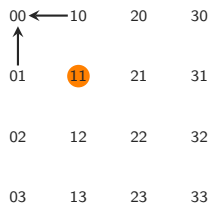
Identify an intersection by the **representative pair** $(\bullet, \bullet)$:
Union nodes which map to the same pair



U



V

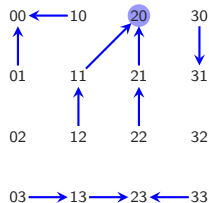


$U \sqcup V$

$(00, 10) \mapsto 00$
 $(20, 30) \mapsto 20$
 $(31, 30) \mapsto 30$
 $(20, 31) \mapsto 11$

Joining union-finds

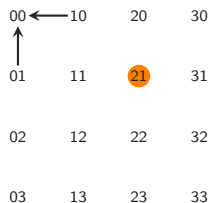
Identify an intersection by the **representative pair** $(\bullet, \bullet)$:
Union nodes which map to the same pair



U



V

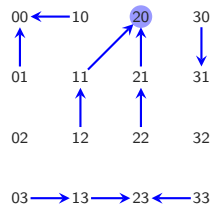


$U \sqcup V$

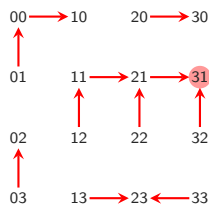
$(00, 10) \mapsto 00$
 $(20, 30) \mapsto 20$
 $(31, 30) \mapsto 30$
 $(20, 31) \mapsto 11$

Joining union-finds

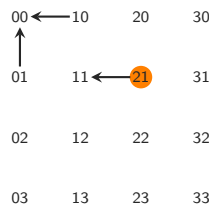
Identify an intersection by the **representative pair** $(\bullet, \bullet)$:
Union nodes which map to the same pair



U



V



$U \sqcup V$

$(00, 10) \mapsto 00$
 $(20, 30) \mapsto 20$
 $(31, 30) \mapsto 30$
 $(20, 31) \mapsto 11$

Almost-linear join

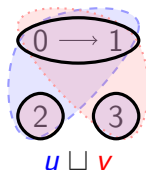
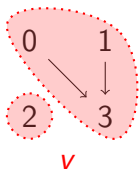
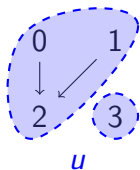
```
let join : UF → UF → UF  $\triangleq$  fun u v  $\mapsto$ 
  let new_cls  $\triangleq$  Hashtbl.new () in
  let res : UF  $\triangleq$  UF.make () in
  for x in nodes do
    let r_u  $\triangleq$  find u x and r_v  $\triangleq$  find v x in
    match Hashtbl.lookup new_cls (r_u, r_v) with
    | Some r_j  $\mapsto$  union res x r_j
    | None  $\mapsto$  new_cls [(r_u, r_v)] := x
  done; return res
```

Amortized complexity

$\mathcal{O}(\#nodes \times \alpha(\#nodes))$ with α the inverse Ackermann function

Join is (almost) asymptotically optimal

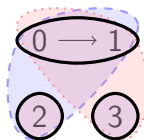
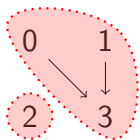
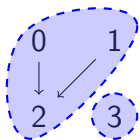
Worst-case join requires $\Omega(\#nodes)$ writes (from either u , v , or empty):



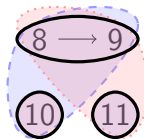
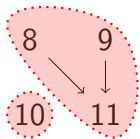
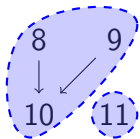
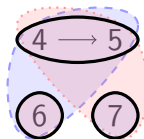
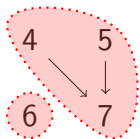
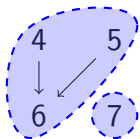
2 writes from u or v
1 write from empty

Join is (almost) asymptotically optimal

Worst-case join requires $\Omega(\#nodes)$ writes (from either u , v , or empty):



$2k$ writes from u or v
 $1k$ writes from empty



Total of $4k$ nodes



Difference-based join

Most joins occur between similar states (small conditional).

Idea: skip nodes with same parents in **u** and **v**

Difference-based join

Most joins occur between similar states (small conditional).

Idea: skip nodes with same parents in u and v

Difference operator

Function $\text{diff} : UF \rightarrow UF \rightarrow \text{node set}$ such that:

$$\forall x : \text{node}, \quad u.\text{parent}[x] \neq v.\text{parent}[x] \Rightarrow x \in \text{diff } u \ v$$

Difference-based join

Most joins occur between similar states (small conditional).

Idea: skip nodes with same parents in u and v

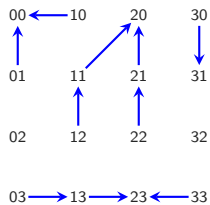
Difference operator

Function $\text{diff} : UF \rightarrow UF \rightarrow \text{node set}$ such that:

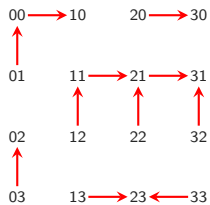
$$\forall x : \text{node}, \quad u.\text{parent}[x] \neq v.\text{parent}[x] \Rightarrow x \in \text{diff } u \ v$$

Must be faster than $\mathcal{O}(\#nodes)$ to be useful

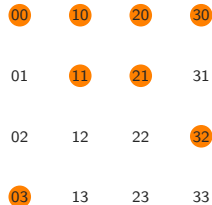
Example difference



U



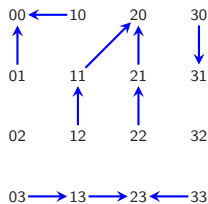
V



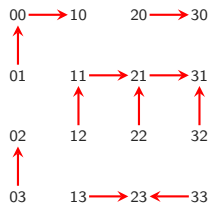
diff *U V*

Difference-based join

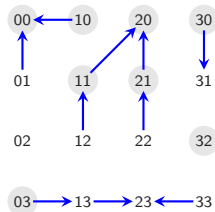
1. start from u instead of empty



U



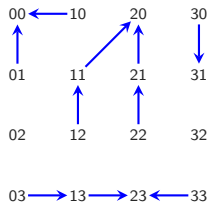
V



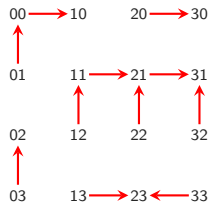
$U \sqcup V$

Difference-based join

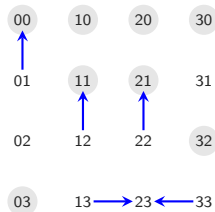
1. start from u instead of empty
2. remove all edges from diff $u \vee$



u



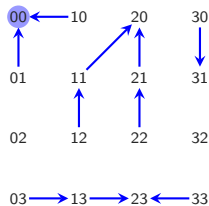
v



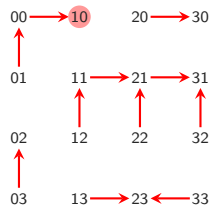
$u \sqcup v$

Difference-based join

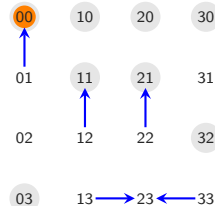
1. start from u instead of empty
2. remove all edges from diff $u \vee$
3. proceed as before*, only iterating the diff $u \vee$



u



v



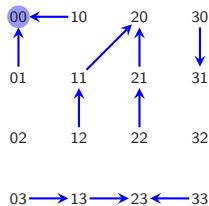
$u \sqcup v$

$(00, 10) \mapsto 00$

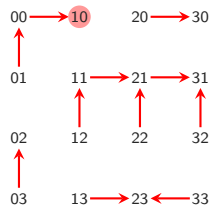
* The mapping must be initialized so that $(x, x) \mapsto x$

Difference-based join

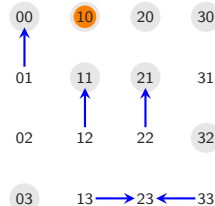
1. start from u instead of empty
2. remove all edges from diff $u \vee$
3. proceed as before*, only iterating the diff $u \vee$



U



V



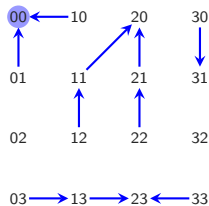
$U \sqcup V$

$(00, 10) \mapsto 00$

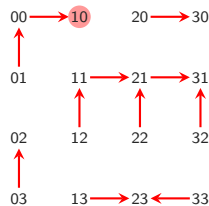
* The mapping must be initialized so that $(x, x) \mapsto x$

Difference-based join

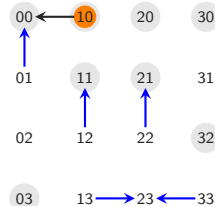
1. start from u instead of empty
2. remove all edges from diff $u \vee$
3. proceed as before*, only iterating the diff $u \vee$



u



v



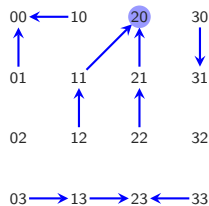
$u \sqcup v$

$(00, 10) \mapsto 00$

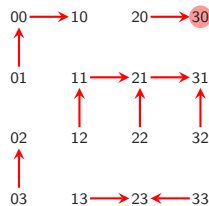
* The mapping must be initialized so that $(x, x) \mapsto x$

Difference-based join

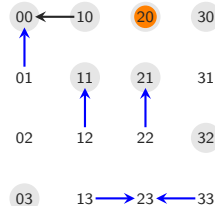
1. start from u instead of empty
2. remove all edges from diff $u \vee$
3. proceed as before*, only iterating the diff $u \vee$



U



V



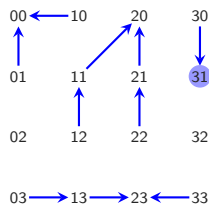
$U \sqcup V$

$(00, 10) \mapsto 00$
 $(20, 30) \mapsto 20$

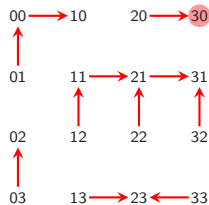
* The mapping must be initialized so that $(x, x) \mapsto x$

Difference-based join

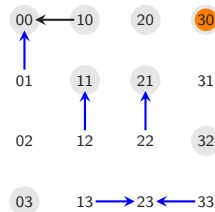
1. start from u instead of empty
2. remove all edges from diff $u \vee$
3. proceed as before*, only iterating the diff $u \vee$



U



V



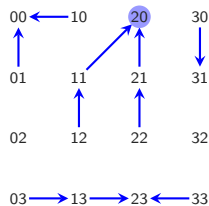
$U \sqcup V$

$(00, 10) \mapsto 00$
 $(20, 30) \mapsto 20$
 $(31, 30) \mapsto 30$

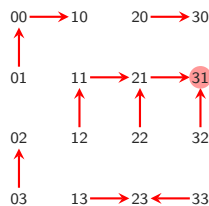
* The mapping must be initialized so that $(x, x) \mapsto x$

Difference-based join

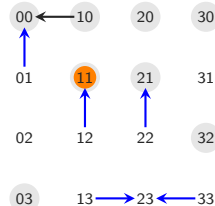
1. start from u instead of empty
2. remove all edges from diff $u \vee$
3. proceed as before*, only iterating the diff $u \vee$



u



v



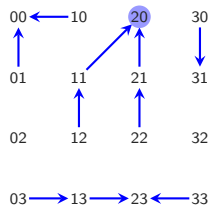
$u \sqcup v$

$(00, 10) \mapsto 00$
 $(20, 30) \mapsto 20$
 $(31, 30) \mapsto 30$
 $(20, 31) \mapsto 11$

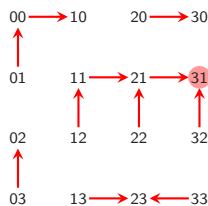
* The mapping must be initialized so that $(x, x) \mapsto x$

Difference-based join

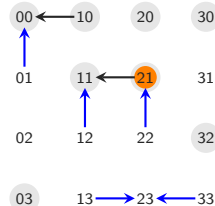
1. start from u instead of empty
2. remove all edges from diff $u \vee$
3. proceed as before*, only iterating the diff $u \vee$



u



v



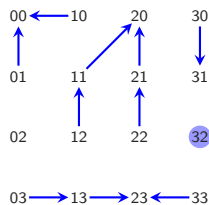
$u \sqcup v$

$(00, 10) \mapsto 00$
 $(20, 30) \mapsto 20$
 $(31, 30) \mapsto 30$
 $(20, 31) \mapsto 11$

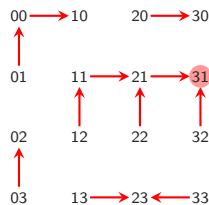
* The mapping must be initialized so that $(x, x) \mapsto x$

Difference-based join

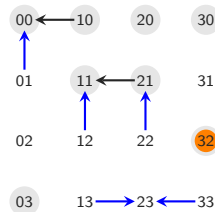
1. start from u instead of empty
2. remove all edges from diff $u \vee$
3. proceed as before*, only iterating the diff $u \vee$



u



v



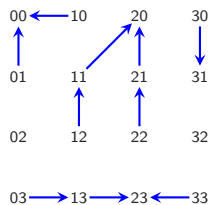
$u \sqcup v$

$(00, 10) \mapsto 00$
 $(20, 30) \mapsto 20$
 $(31, 30) \mapsto 30$
 $(20, 31) \mapsto 11$
 $(32, 31) \mapsto 32$

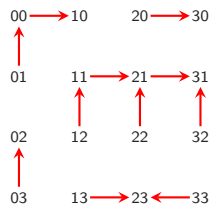
* The mapping must be initialized so that $(x, x) \mapsto x$

Difference-based join

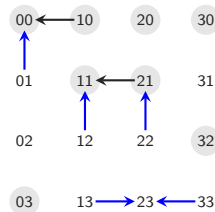
1. start from u instead of empty
2. remove all edges from diff $u \vee$
3. proceed as before*, only iterating the diff $u \vee$



U



V



$U \sqcup V$

$(00, 10) \mapsto 00$
 $(20, 30) \mapsto 20$
 $(31, 30) \mapsto 30$
 $(20, 31) \mapsto 11$
 $(32, 31) \mapsto 32$

Amortized complexity: $\mathcal{O}(\delta \times \alpha(\#nodes))$

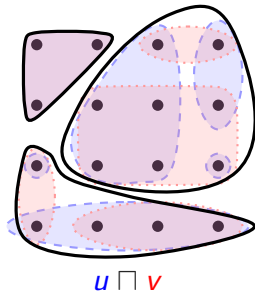
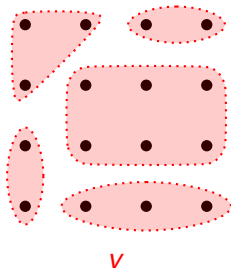
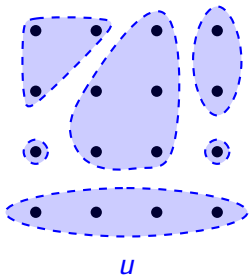
* The mapping must be initialized so that $(x, x) \mapsto x$

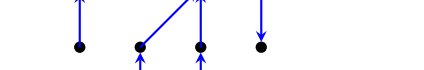


2. Meet and inclusion

Meeting union-finds

Meet is **equivalence class union**:

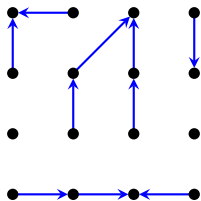




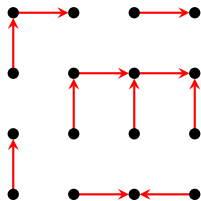
Meeting union-finds

Meet is **equivalence class union**:

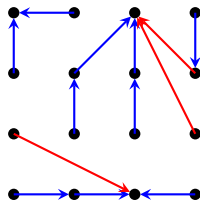
1. Start from **u**
2. Add equalities from **v** (union $x \equiv_v y$ for $x \equiv_v y$)



U



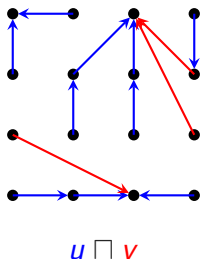
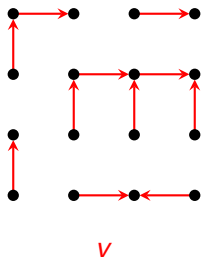
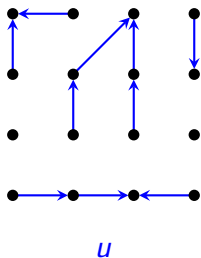
V



U ∩ V

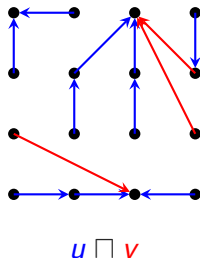
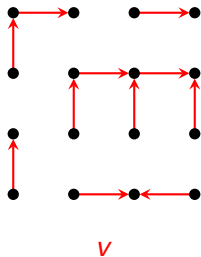
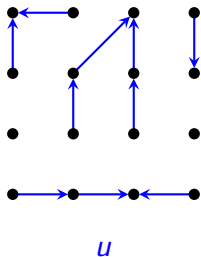
Meet is **equivalence class union**:

1. Start from u
2. Add *parent arrows* from v ($\text{union } x \ v.\text{parent}[x]$ for $x \in \text{node}$)



Meet is **equivalence class union**:

1. Start from u
2. Add *parent arrows* from v (union x $v.parent[x]$ for $x \in \text{diff } u \ v$)



Meeting union-finds

```
let meet : UF → UF → UF  $\triangleq$  fun u v  $\mapsto$   
    diff u v  $\triangleright$  fold (fun res x  $\mapsto$  union res x v.parent[x]) u
```

Amortized complexity

$\mathcal{O}(\delta \times \alpha(\text{\#nodes}))$

Union-find inclusion

Definition: $u \sqsubseteq v \Leftrightarrow$ all equalities from v hold in u

Union-find inclusion

Definition: $u \sqsubseteq v \Leftrightarrow$ all equalities from v hold in u

Simplification: It suffices to check $x \equiv_u v.parent[x]$ for all x

Union-find inclusion

Definition: $u \sqsubseteq v \Leftrightarrow$ all equalities from v hold in u

Simplification: It suffices to check $x \equiv_u v.parent[x]$ for all x in $\text{diff } u \ v$

Union-find inclusion

Definition: $u \sqsubseteq v \Leftrightarrow$ all equalities from v hold in u

Simplification: It suffices to check $x \equiv_u v.parent[x]$ for all x in $\text{diff } u \ v$

```
let incl : UF → UF → bool  $\triangleq$  fun u v  $\mapsto$   
  diff u v  
  ▷ for_all (fun x  $\mapsto$  find u x = find u v.parent[x])
```

Amortized complexity

$\mathcal{O}(\delta \times \alpha(\#nodes))$



3. Implementing the difference operator

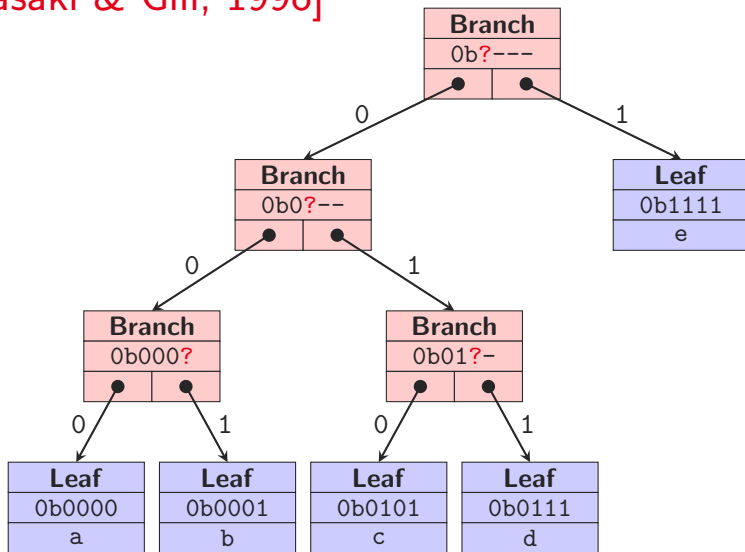
Patricia tree [Okasaki & Gill, 1998]

Binary prefix trie

Can represent maps:

$0 \mapsto a$
 $1 \mapsto b$
 $5 \mapsto c$
 $7 \mapsto d$
 $15 \mapsto e$

Shape only depends
on contents, not
insertion order



Patricia-tree-based union-find

Fast intersection iterator: by iterating two trees simultaneously

- Skip shared subtrees
- Skip subtrees that appear only in one argument

Fast union iterator: skip shared subtrees

Patricia-tree-based union-find

Fast intersection iterator: by iterating two trees simultaneously

- Skip shared subtrees
- Skip subtrees that appear only in one argument

Fast union iterator: skip shared subtrees

Pros

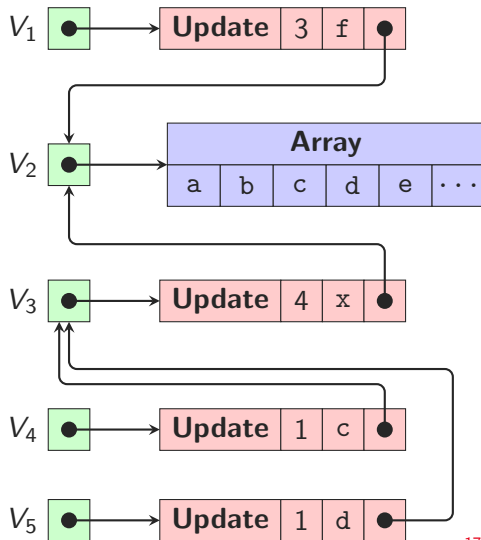
- + Truly immutable, thread-safe
- + Sparse representation
- + Join skips branch terms

Cons

- $\mathcal{O}(\log \# \text{unions})$ access cost

Persistent arrays [Conchon & Filliâtre, 2007]

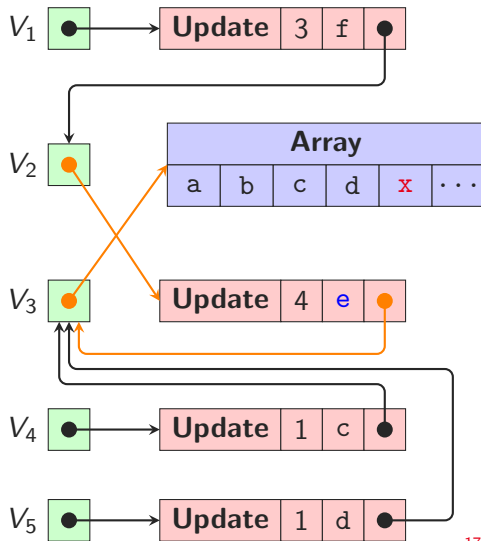
Version pointers to one live array or an update of another version.



Persistent arrays [Conchon & Filliâtre, 2007]

Version pointers to one live array or an update of another version.

Access to another version (V_3) moves the array there

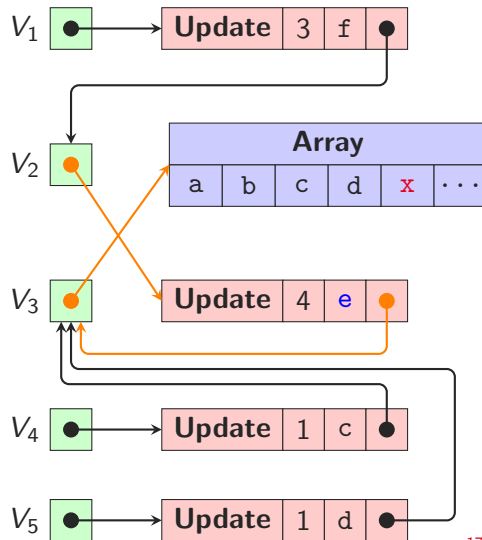


Persistent arrays [Conchon & Filliâtre, 2007]

Version pointers to one live array or an update of another version.

Access to another version (V_3) moves the array there

Re-accessing the same version is cheap, changing versions is costly.





Persistent-array-based union-find

Difference: follow pointers between the two versions, accumulating keys

Reschedule algorithms to limit re-roots.

Persistent-array-based union-find

Difference: follow pointers between the two versions, accumulating keys

Reschedule algorithms to limit re-roots.

Pros

- + $\mathcal{O}(1)$ access cost
- + Better cache locality

Cons

- Must limit version switches
- Interior mutability (not thread-safe)
- No inclusion early return

Comparing algorithms

Variant	join	meet	$\sqsubseteq$ (true)	$\sqsubseteq$ (false)
Naive class intersection	9.9e5	—	28	0.009
Almost linear	114	7.3	2.9	0.002
Diff / Patricia tree	4.5	7.1	2.1	0.002
Diff / Persistent array	9.2	7.3	1.3	6.4

Runtimes in ms, 250k nodes, 50k unions, 5k difference

Comparing algorithms

Variant	join	meet	$\sqsubseteq$ (true)	$\sqsubseteq$ (false)
Naive class intersection	9.9e5	—	28	0.009
Almost linear	114	7.3	2.9	0.002
Diff / Patricia tree	4.5	7.1	2.1	0.002
Diff / Persistent array	9.2	7.3	1.3	6.4

Runtimes in ms, 250k nodes, 50k unions, 5k difference

Used in Gammalphi: compared to non-relational SSA analysis

- 19 precision gains / 638 files

Comparing algorithms

Variant	join	meet	$\sqsubseteq$ (true)	$\sqsubseteq$ (false)
Naive class intersection	9.9e5	—	28	0.009
Almost linear	114	7.3	2.9	0.002
Diff / Patricia tree	4.5	7.1	2.1	0.002
Diff / Persistent array	9.2	7.3	1.3	6.4

Runtimes in ms, 250k nodes, 50k unions, 5k difference

Used in Gammalphi: compared to non-relational SSA analysis

- 19 precision gains / 638 files
- Diff / Patricia tree add 5-10% cost overhead
- Diff / Persistent array add 300-400% cost overhead (and many timeouts)



4. Conclusion

Conclusion



Contributions:

- Algorithms for join, meet, and inclusion test on union-find
- Good performance by operating on difference: $\mathcal{O}(\delta \times \alpha(\#\text{nodes}))$
- Two data structures for difference: Patricia trees and persistent array
- Experiments validate performance

Available as an OCaml library: `union-find-lattice`



Paper



Library